

SPEC.md — DigitalAsset Guard AI Copilot

Bản đặc tả duy nhất (Single Source of Truth) — dùng để build lại dự án từ đầu

File này là nguồn tài liệu **duy nhất** dùng khi làm việc với AI code. Không đưa PDF gốc, `phan_sua_doi.md`, hay repo cũ cho AI nữa — mọi thứ cần thiết đã gộp vào đây. Khi yêu cầu AI viết module, chỉ trích đúng mục liên quan trong file này.

0. Tóm tắt 2 phút (Executive Summary)

DigitalAsset Guard AI Copilot là trợ lý AI đa tác nhân (Multi-Agent) hỗ trợ chuyên viên tuân thủ ngân hàng Việt Nam điều tra rủi ro dòng tiền tài sản số (on-chain), đối chiếu dữ liệu nội bộ (off-chain), và soạn thảo báo cáo giao dịch đáng ngờ (STR) — theo cơ chế con người phê duyệt cuối cùng (Human-in-the-loop).

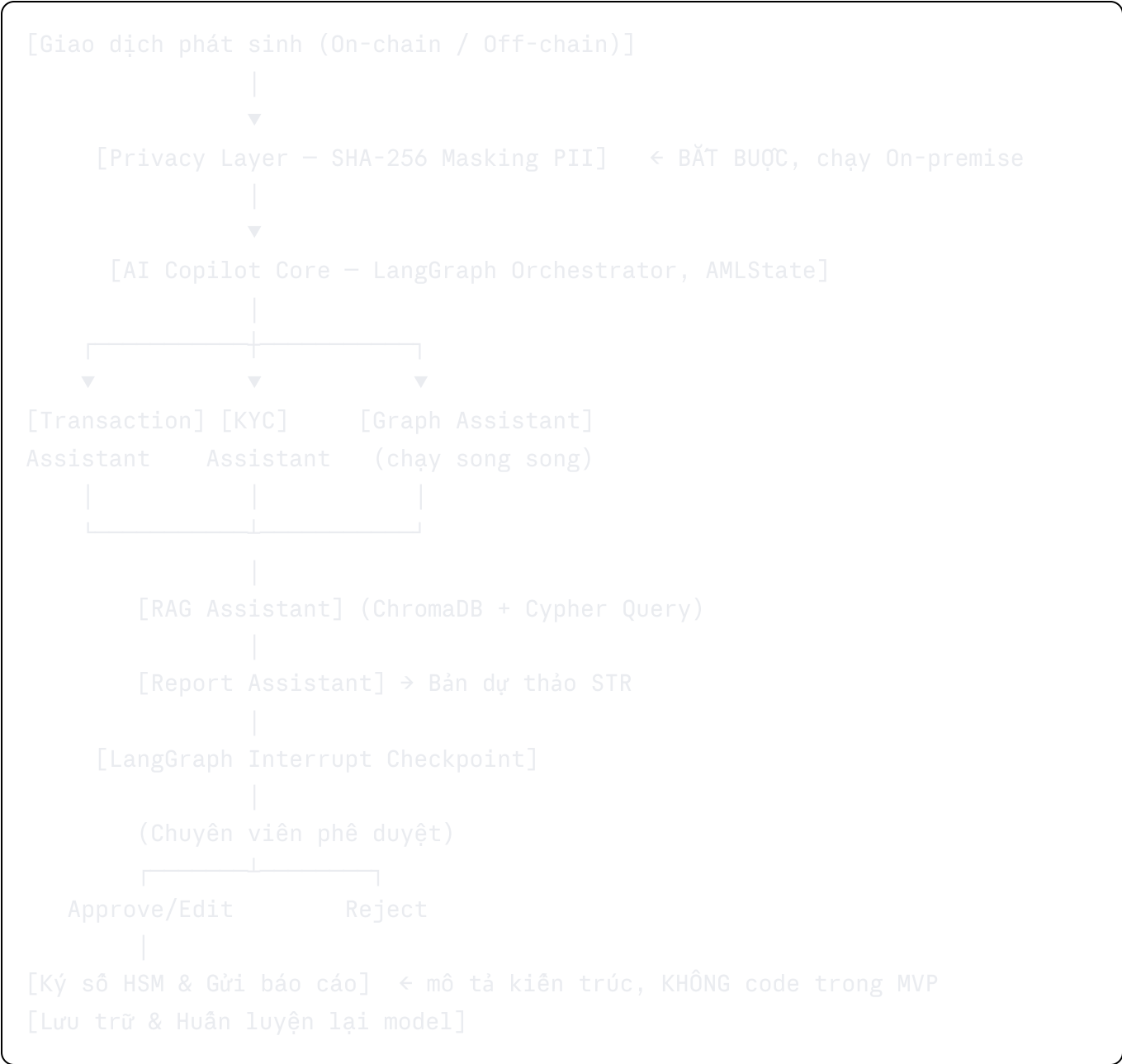
Pain point cốt lõi: hệ thống rule-based cũ có tỷ lệ báo động giả (False Positives) tới 95%; chuyên viên AML mất 4-6 giờ/hồ sơ để tra cứu thủ công qua hàng chục công cụ rời rạc (blockchain explorer, danh sách trừng phạt OFAC, văn bản luật, Core Banking).

Mục tiêu: giảm 60% thời gian điều tra (4h → 1.5h), hạ False Positives xuống dưới 50%, tự động hóa soạn thảo STR, và tính đúng nghĩa vụ thuế TNCN 0.1% cho giao dịch tài sản số.

Bối cảnh pháp lý bắt buộc tuân thủ:

- Thông tư 27/2025/TT-NHNN — báo cáo giao dịch đáng ngờ, ngưỡng báo cáo (500 triệu VND trong nước / 1.000 USD quốc tế), hạn nộp báo cáo 24h.
- Thông tư 32/2026/TT-BTC — thuế TNCN 0.1% trên giao dịch tài sản mã hóa.
- Nghị định 13/2023/NĐ-CP — bảo vệ dữ liệu cá nhân (PII).
- Nghị định 23/2025/NĐ-CP — giá trị pháp lý chữ ký số (HSM) — **không thuộc phạm vi code MVP**, chỉ mô tả ở tầng workflow.

1. Kiến trúc tổng quan



Điểm truy cập bổ sung (không thuộc luồng xử lý on-chain): một **FastAPI Gateway tối giản** chạy song song với UI Streamlit, phục vụ dòng doanh thu API-as-a-Service (§7 dưới đây). Gateway này expose 1 endpoint tra cứu địa chỉ ví (sàng lọc OFAC/UN/NHNN + risk score sơ bộ XGBoost), tách biệt khỏi luồng LangGraph đầy đủ.

Giải thích các thành phần cốt lõi

Thành phần	Vai trò	Ghi chú kỹ thuật
Transaction Entry	Nhận dữ liệu giao dịch VNĐ (Core Banking) + crypto (API blockchain)	—
Privacy Layer	Băm SHA-256 toàn bộ PII on-premise trước khi vào AI Core	Xem §4 quy ước đặt tên
AI Copilot Core	Điều phối tuần tự/tắt định qua LangGraph, giữ state AMLState	Finite State Machine
5 Assistants	Xem §3	—
Human Review Checkpoint	LangGraph interrupt, dừng chờ duyệt	Bắt buộc theo Thông tư 27
STR Submission (ký số HSM)	Mô tả kiến trúc, gửi báo cáo về Cục PCRT	Không thuộc MVP Real Components

2. Workflow xử lý (8 bước)

- Webhook giao dịch** — kích hoạt khi giao dịch vượt ngưỡng báo cáo (500tr VND / 1.000 USD).
- Ẩn danh hóa PII (Privacy Layer)** — băm SHA-256 tại chỗ, chỉ metadata kỹ thuật + ví ẩn danh được chuyển tiếp.
- Gán nhãn rủi ro ban đầu** — Transaction Assistant, mô hình XGBoost.
- Sàng lọc danh sách trừng phạt** — KYC Assistant, so khớp mờ với OFAC SDN / UN / NHNN.
- Phân tích đồ thị dòng tiền** — Graph Assistant, Neo4j (PPR + Louvain).
- Đối chiếu pháp lý & tính thuế** — RAG Assistant, trích dẫn Thông tư 27 và Thông tư 32.
- Tạm dừng chờ duyệt (HITL)** — Report Assistant tổng hợp bản dự thảo STR, LangGraph interrupt.
- Ký số & gửi báo cáo** — chuyên viên Approve → HSM ký số → gửi Cục PCRT trong 24h. (Mô tả kiến trúc, không code trong MVP)

3. Kiến trúc AI — Chi tiết 5 Trợ lý (5 Assistants)

3.1 Transaction Assistant

- Tiếp nhận dữ liệu kỹ thuật ẩn danh (đã qua Privacy Layer).

- Mô hình **XGBoost** huấn luyện trên **Elliptic Bitcoin Dataset** (203.769 nút, 234.355 cạnh, tỷ lệ illicit ~2%).
- Gán nhãn rủi ro ban đầu dựa trên ngưỡng Thông tư 27 (500tr VND).
- **Đánh giá mô hình:** ưu tiên Recall, F1-score, và **AUC-PR** (không phải AUC-ROC) do dữ liệu mất cân bằng nghiêm trọng. Bảng số liệu bắt buộc phải điền số thật khi báo cáo:

Chỉ số	Ghi chú
Accuracy	Chỉ tham khảo, không dùng đánh giá chính (do mất cân bằng nhãn)
Precision (illicit)	Tỷ lệ cảnh báo đúng
Recall (illicit)	Chỉ số quan trọng nhất
F1-score (illicit)	Cân bằng Precision/Recall
AUC-PR	Phù hợp hơn AUC-ROC khi dữ liệu mất cân bằng

- **Chia tập train/test:** bắt buộc dùng **temporal split** theo `time_step` gốc của Elliptic (`time_step ≤ 34` → train, `> 34` → test). **Không dùng random split** — tránh rò rỉ dữ liệu tương lai vào quá khứ.
- Loại bỏ nhãn "unknown" khỏi tập huấn luyện/đánh giá, chỉ giữ licit(2)/illicit(1).
- Dùng `scale_pos_weight` để bù mất cân bằng lớp.

3.2 KYC Assistant

- Đối chiếu địa chỉ ví với danh sách trừng phạt OFAC SDN (hơn 6.000 thực thể), UN, NHNN.
- Thuật toán so khớp mờ (Fuzzy matching — Levenshtein Distance) + gom cụm ví (wallet clustering).
- **Định dạng dữ liệu OFAC: XML** (`sdn.xml`), xử lý bằng `xml.etree.ElementTree.iterparse` để tránh tràn RAM (OOM) trên máy cá nhân. **Không phải file text thô.**

3.3 Graph Assistant

- Xây dựng đồ thị tri thức giao dịch đa hop trên **Neo4j** (Production) / **NetworkX** (Demo Mode).
- **Personalized PageRank (PPR):** lan truyền rủi ro từ ví đen OFAC sang ví lân cận.
 - Công thức: $PR_s(u) = (1-d) \cdot e_s(u) + d \cdot \sum (PR_s(v) / L(v))$ với $v \in B_u$
 - $[d]$ = damping factor (mặc định 0.85); $[e_s]$ = vector cá nhân hóa tập trung vào ví nguồn độc hại đã biết.

- **Louvain Community Detection:** tối đa hóa Modularity Q để phát hiện "đảo gian lận" (fraud rings).

- Công thức: $Q = (1/2m) \cdot \sum [A_{ij} - k_i \cdot k_j / 2m] \cdot \delta(c_i, c_j)$

3.4 RAG Assistant

- Chạy truy vấn Cypher trên Neo4j để lấy đặc trưng cấu trúc (PPR score, ID cộng đồng Louvain).
- Mã hóa thành văn bản ngữ cảnh, ghép vào tìm kiếm vector trên **ChromaDB**.
- Trích xuất căn cứ pháp lý từ **Thông tư 27 (AML)** và **bắt buộc phải có Thông tư 32/2026/TT-BTC** (thuế TNCN 0.1% tài sản số) — đây là lỗ hổng dữ liệu đã phát hiện, phải có file tri thức thô của Thông tư 32 trong `data/legal_docs/`, không chỉ Thông tư 27.

3.5 Report Assistant

- Tổng hợp phân tích kỹ thuật + lập luận pháp lý thành **Weighted Risk Score**:

$$\text{Final Score} = 0.2 \times \text{Classifier Score} + 0.3 \times \text{KYC Flags} + 0.5 \times \text{Graph Risk}$$

Lý do chọn trọng số: Graph Risk được đánh giá cao nhất (0.5) vì phản ánh hành vi lan truyền qua nhiều hop giao dịch, khó giả mạo hơn kiểm tra đơn lẻ; KYC Flags (0.3) là tín hiệu trực tiếp từ danh sách trừng phạt; Classifier Score (0.2) chỉ là chấm điểm sơ bộ ban đầu, độ tin cậy thấp nhất trong 3 tín hiệu. (Ghi chú: công thức này không có trong báo cáo khả thi gốc — là quyết định kỹ thuật bổ sung khi code, cần được note lại rõ ràng để giải trình khi bảo vệ.)

- Nếu Final Score $\geq 0.7 \rightarrow$ biên soạn bản dự thảo báo cáo STR (Mẫu số 04, Thông tư 27) bằng `python-docx`, lưu vào `reports/output/STR_REPORT_<tx_hash>.docx`.
- Đính kèm sơ đồ dòng tiền trực quan từ Neo4j.
- Kích hoạt LangGraph interrupt chờ phê duyệt.

4. Privacy Layer — Quy ước bắt buộc

Đây là điểm bổ sung quan trọng nhất so với repo v1 (thiếu hoàn toàn). Đáp ứng Nghị định 13/2023/NĐ-CP và là USP #2 của sản phẩm so với Chainalysis/Elliptic/TRM Labs.

- Module: `core/privacy_layer.py`. Chạy **on-premise**, băm **SHA-256** (có salt cấu hình qua `.env`) cho mọi trường PII **trước khi** dữ liệu vào AI Copilot Core.
- **Quy ước đặt tên bắt buộc** (để Agent không đọc nhầm dữ liệu gốc):

Trường gốc	Trường sau băm
fullname	hashed_fullname
id_number (CCCD)	hashed_id_number
account_number	hashed_account_number

- `AMLState` (`core/state.py`) chỉ được lưu các trường có tiền tố `hashed_`. Không lưu, không log, không truyền bản PII gốc dưới bất kỳ hình thức nào ra khỏi Privacy Layer.
- Toàn bộ Agent từ Transaction Assistant đến Report Assistant (bước 4-8 trong lộ trình build) chỉ được phép đọc trường có tiền tố `hashed_`. Đây là ràng buộc cứng khi code — nếu một agent cần "tên khách hàng" để hiển thị, phải xử lý ở tầng UI/Report cuối cùng bằng cách tra ngược qua bảng ánh xạ được bảo vệ riêng, không đưa PII gốc vào state chung.
- Test bắt buộc: (a) dữ liệu PII gốc không xuất hiện trong state truyền cho agent; (b) cùng input → cùng hash (deterministic); (c) input khác nhau → hash khác nhau.

5. MVP — Real Components vs Mock Components

Bản thử nghiệm chạy trên máy cá nhân (GTX 1650 / RTX 3060 Ti), phạm vi cho cuộc thi Fintech ATTACKER 2026.

Real Components (chạy thật)

- Crawl blockchain: API Etherscan (miễn phí, qua `requests`) → Neo4j. (BscScan: xem quyết định cần chốt ở §8, mục "Cần bạn xác nhận")
- Tra cứu danh sách đen: parse `sdn.xml` bằng `xml.etree.ElementTree.iterparse` (không OOM).
- Thuật toán đồ thị: Neo4j Community Edition local, Cypher + PPR + Louvain.
- Chấm điểm rủi ro: XGBoost huấn luyện trên Elliptic (CPU), **temporal split**, đánh giá bằng **AUC-PR**.
- RAG luật & thuế: ChromaDB local + API OpenAI/Gemini, nguồn gồm **Thông tư 27** và **Thông tư 32**.
- Điều phối đa tác nhân: LangGraph, schema `AMLState`, rẽ nhánh bằng Python thuần.
- Tự động tạo báo cáo: `python-docx`, mẫu biểu Thông tư 27.
- **Privacy Layer**: SHA-256 masking on-premise (bổ sung mới — §4).
- **API Gateway**: FastAPI tối giản, 1 endpoint tra cứu ví (screening + risk score sơ bộ), phục vụ demo gói API-as-a-Service.

Mock Components (mô phỏng)

- Dữ liệu ngân hàng: JSON/CSV giả lập tài khoản off-chain.

- LLM: dùng API cloud OpenAI/Gemini thay Local LLM (sản xuất thực tế mới cần Local LLM theo Nghị định 13).
- Xử lý thời gian thực: batch/simulate-stream, chưa dùng Kafka.
- **Ký số HSM & gửi báo cáo NHNN:** chỉ mô tả ở tầng workflow/UI, không code kết nối HSM thật.

6. Cấu trúc thư mục (giữ nguyên từ v1, cập nhật OFAC format)

text

```
digitalasset_guard/
├── .env
├── .gitignore
├── requirements.txt
├── SPEC.md # ← file này, thay thế README cũ làm nguồn sự t
├── demo_run.py
├── agents/
│   ├── __init__.py
│   ├── transaction_classifier.py
│   ├── train_classifier.py # temporal split + scale_pos_weight
│   ├── kyc_verification.py # đọc sdn.xml qua iterparse
│   ├── graph_aml.py
│   ├── regulation_rag.py # nguồn: Thông tư 27 + Thông tư 32
│   └── alert_report.py # Weighted Risk Score có docstring giải thích t
├── api/ # MỞI – FastAPI Gateway cho API-as-a-Service
│   └── main.py
├── core/
│   ├── __init__.py
│   ├── config.py
│   ├── state.py # chỉ chứa trường hashed_*
│   ├── privacy_layer.py # MỞI – SHA-256 masking
│   └── graph_builder.py
├── data/
│   ├── legal_docs/
│   │   ├── fatf_recommendations.txt
│   │   ├── thông_tu_27_2025.txt
│   │   └── thông_tu_32_2026.txt # MỞI – bắt buộc, thuế TNCN 0.1%
│   ├── mock/
│   │   ├── customers.json
│   │   └── generate_mock_banking.py
│   ├── processed/
│   │   └── sample_ofac_wallet.txt
│   └── raw/
│       ├── elliptic/
│       ├── etherscan/
│       └── ofac/
│           └── sdn.xml # SỬA – định dạng XML, không phải .txt
├── db/
│   ├── neo4j_setup.py
│   └── vector_db.py
├── models/
│   └── xgboost_aml.pkl
├── reports/
```



```

|   ├── templates/
|   └── output/
├── scripts/
|   ├── 00_healthcheck.py
|   ├── 01_check_ofac.py
|   ├── 02_fetch_etherscan_sample.py
|   └── 03_load_elliptic.py
├── tests/
|   ├── __init__.py
|   ├── evaluate_model.py      # SỬA – Recall/F1/AUC-PR thay vì chỉ ROC-AUC
|   └── test_agents.py         # + test cho privacy_layer
└── ui/
    └── app.py                  # Streamlit

```

7. Kế hoạch kinh doanh (tóm tắt, không cần code tương ứng)

- **Roadmap 3 giai đoạn:** Năm 1 (2-3 khách FinTech/VASP thử nghiệm) → Năm 2 (5 ngân hàng tầm trung, pilot sandbox NHNN) → Năm 3 (15 ngân hàng lớn/quốc doanh, thương mại hóa toàn diện).
- **4 dòng doanh thu:**
 1. SaaS: 2.000 USD/tháng, tối đa 10.000 ví giám sát.
 2. Enterprise License: 150.000 USD một lần + 30.000 USD/năm bảo trì (từ năm 2).
 3. Professional Services: Integration 20.000 USD/dự án + Training 5.000 USD.
 4. **API-as-a-Service** (mới): 0.05-0.15 USD/lượt tra cứu ví, không phí cố định — dùng chung endpoint FastAPI Gateway ở §5/§6.
- **Phân khúc khách hàng mở rộng:** VASP/sàn giao dịch, công ty bảo hiểm/chứng khoán, ví điện tử, công ty kiểm toán/hãng luật (kênh reseller).
- **Đối thủ cạnh tranh:** Chainalysis, Elliptic (công ty phân tích, không liên quan tên dự án), TRM Labs. USP: (1) bản địa hóa pháp lý VN sâu (Thông tư 27+32), (2) Privacy Layer SHA-256 on-premise, (3) chi phí phù hợp ngân hàng vừa/VASP khởi nghiệp.
- **Rủi ro Go-to-market:** chu kỳ bán hàng Enterprise B2B 12-18 tháng; SaaS/API-as-a-Service (1-3 tháng) tạo dòng tiền đệm.

8. Việc cần bạn xác nhận trước khi build (chưa chốt)

- ☐ **BscScan trong MVP:** có triển khai crawl BNB Chain thật, hay chỉ demo Ethereum và ghi chú BscScan ở "Hướng phát triển"? (Khuyến nghị: chỉ Ethereum cho MVP, thêm BscScan sau nếu còn thời gian.)
- ☐ Salt dùng cho SHA-256 trong Privacy Layer: cố định trong `.env` hay xoay vòng theo phiên? (Khuyến nghị: cố định trong `.env`, đơn giản cho MVP, ghi rõ giới hạn này trong phần

9. Thứ tự build (nhắc lại từ lộ trình reset — dùng khi giao việc cho AI)

1. `core/config.py`, `core/state.py`
2. `core/privacy_layer.py`
3. `data/` + `scripts/` (Etherscan, Elliptic, `sdn.xml`)
4. `agents/transaction_classifier.py` + `train_classifier.py` (temporal split, AUC-PR)
5. `agents/kyc_verification.py` (đọc XML qua iterparse)
6. `agents/graph_aml.py`
7. `agents/regulation_rag.py` + `db/vector_db.py` (nguồn: TT27 + TT32)
8. `agents/alert_report.py` (Weighted Risk Score có docstring)
9. `core/graph_builder.py`
10. `api/main.py` (FastAPI Gateway) + `ui/app.py` (Streamlit) + `demo_run.py`

Mẫu prompt giao việc cho AI ở mỗi bước:

"Đây là SPEC.md của dự án. Hệ thống đã build xong đến bước [X]. Hãy viết ĐỌC LẬP module [Y] theo đúng đặc tả tại mục [số mục trong SPEC.md]. Không tự ý sửa file cũ; nếu cần gọi hàm từ module cũ thì import vào. Nếu cần gọi module chưa có, dùng dữ liệu MOCK và ghi TODO."

Sau mỗi module: chạy thử độc lập (`python -m agents.<module>`), xác nhận output đúng kỳ vọng rồi mới sang bước kế tiếp.